



HACK THE BOX · LINUX

Codify

Informe tècnic complet ·
explotació reproduïble

Màquina Codify resolta

HTB Codify · 06 de maig de 2026

Màquina	Codify
Plataforma	Hack The Box
Estat	Retired
Dificultat	Easy
Sistema	Linux
Data	06 de maig de 2026
Autor	Ramon Santos Sabarich / r4ms4nt



Informe tècnic normalitzat per a
arxivament, consulta i traçabilitat.

GUIA DE LECTURA

Índex

Informe tècnic didàctic de Codify. L'estructura conserva la seqüència real del laboratori: enumeració inicial, identificació de la superfície dominant, explotació, moviment lateral, escalada de privilegis i tancament tècnic.

Informe tècnic didàctic

1. Sinopsi tècnica
2. Preparació i arrencada del laboratori
 - 2.1 Ús de Inici-HTB
 - 2.2 Evidència inicial
 - 2.3 Lectura de la fase 1
3. Normalització del hostname i comparació de superfícies HTTP
 - 3.1 Resolució de codify.htb
 - 3.2 Comparació entre IP, hostname i port 3000
 - 3.3 Extracció de rutes públiques
 - 3.4 Tancament de WEB-BASE inicial
4. Identificació de la tecnologia de sandbox
 - 4.1 Descàrrega de pàgines informatives
 - 4.2 Cerca de termes clau
 - 4.3 Restriccions declarades per l'aplicació
 - 4.4 Flux real de l'editor
5. Validació benigna de l'endpoint /run
 - 5.1 Per què validar abans d'explotar
 - 5.2 Prova contra /run
 - 5.3 Validació de mòdul restringit
6. Candidata pública: escape de sandbox a vm2
 - Fet verificat
 - Inferència raonable
 - Pendent fins a validació
7. Validació de l'escape de sandbox
 - 7.1 Prova mínima amb id
 - 7.2 Preparació de la reverse shell
 - 7.3 PoC per obtenir shell
8. Enumeració local després del foothold
 - 8.1 Identificació d'usuaris interactius
 - 8.2 Cerca d'artefactes web
 - 8.3 Confirmació de la base SQLite
9. Extracció de hash des de tickets.db
 - 9.1 Enumeració de taules
 - 9.2 Volcat de la taula users
 - 9.3 Craqueig offline
10. Moviment lateral per SSH
11. Enumeració sudo com joshua
 - 11.1 Revisió de permisos sudo
 - 11.2 Inspecció de l'script abans d'executar-lo
12. Anàlisi de la vulnerabilitat de l'script
 - 12.1 Comparació insegura en Bash
 - 12.2 Exposició de contrasenya per arguments de procés
13. Captura de la contrasenya real amb pspy
 - 13.1 Preparació a la màquina atacant
 - 13.2 Transferència a la víctima
 - 13.3 Segona sessió SSH per disparar l'script
 - 13.4 Contrasenya capturada
14. Escalada final a root

15. Resum tècnic final

15.1 Cadena d'explotació

15.2 Flags

15.3 Credencials i secrets del cas

16. Lliçons reutilitzables

16.1 Llegir l'aplicació abans de buscar exploits

16.2 Distingir bloqueig directe de seguretat real

16.3 Validar el flux abans d'explotar

16.4 L'evidència real corregeix la ruta del writeup

16.5 Les bases locals d'aplicació són pivots freqüents

16.6 El craqueig es fa offline

16.7 No executar scripts sudo sense llegir-los

16.8 Els advertiments importen

16.9 Diferenciar peticions de contrasenya evita errors

16.10 Incidències operatives que no són troballes de la màquina



Hack The Box - Codify

Informe tècnic didàctic

Màquina	Codify
Plataforma	Hack The Box
Sistema	Linux
Dificultat	Easy
Data de treball	2026-05-06, zona Europe/Madrid
Estat	Resolta
Usuari inicial obtingut	svc
Usuari lateral	joshua
Usuari final	root

1. Sinopsi tècnica

Codify exposa una aplicació web desenvolupada amb Node.js/Express que permet executar fragments de codi JavaScript dins d'un entorn de sandbox. La mateixa aplicació documenta l'ús de `vm2` i enllaça la versió `3.9.16`, fet que converteix la superfície web en una via candidata per a un escape de sandbox.

La resolució es basa en una cadena d'evidències progressiva:

1. enumeració inicial amb `Inici-HTB`;
2. identificació d'una aplicació Node.js/Express als ports `80` i `3000`;
3. detecció de `vm2 3.9.16` com a tecnologia de sandbox;
4. validació benigna de l'endpoint `/run`;
5. comprovació del bloqueig directe de mòduls sensibles com `child_process`;
6. explotació d'una vulnerabilitat pública compatible amb la versió observada;
7. obtenció de shell com `svc`;
8. cerca d'artefactes locals de l'aplicació;
9. extracció d'un hash bcrypt de `joshua` des d'una base SQLite;
10. craqueig offline del hash i accés SSH com `joshua`;
11. revisió de permisos `sudo`;
12. anàlisi d'un script de còpia de seguretat executable com `root`;
13. abús d'una comparació insegura en Bash;
14. captura de la contrasenya real observant processos amb `pspy`;
15. autenticació com `root` i lectura de la flag final.

La cadena completa queda resumida així:

2. Preparació i arrencada del laboratori

2.1 Ús de Inici-HTB

El cas va començar amb l'execució de l'script propi **Inici-HTB**, utilitzat per preparar l'entorn de treball, comprovar connectivitat i generar una base inicial d'evidències.

Aquest script no explota res. El seu valor és ordenar el laboratori des de l'inici i deixar una estructura mínima de treball:

La idea didàctica d'aquesta fase és important: abans de buscar vulnerabilitats, cal saber quina superfície real existeix. En una màquina Linux senzilla, una arrencada deficient sol provocar dos errors: saltar a l'explotació sense evidència o perdre temps en branques secundàries.

2.2 Evidència inicial

La màquina va respondre al ping:

```
..._seq=1 ttl=63 time=47.4 ms
```

El TTL 63 i la sortida de `whichSystem.py` apuntaven a Linux:

L'escaneig complet de ports va detectar tres ports oberts:

L'escaneig de serveis va precisar millor la superfície:

A més, el port 80 mostrava una redirecció cap a:

2.3 Lectura de la fase 1

La primera decisió metodològica va ser clara:

SSH va quedar anotat com a via secundària perquè estava obert, però encara no hi havia credencials. La superfície dominant era web, amb dues senyals molt fortes:

3. Normalització del hostname i comparació de superfícies HTTP

3.1 Resolució de codify.htb

Nmap ja havia revelat el hostname `codify.htb`, de manera que es va afegir al fitxer `/etc/hosts`:

Comprovació:

Sortida:

Aquest pas era necessari perquè el port `80` redirigia cap al nom de host. Si la màquina atacant no resolía `codify.htb`, les peticions HTTP no representarien correctament el comportament real de l'aplicació.

3.2 Comparació entre IP, hostname i port 3000

Es van revisar capçaleres HTTP per IP, per hostname i pel port directe `3000`:

La IP redirigia al hostname:

El hostname retornava `200 OK` i mostrava Express darrere d'Apache:

El port `3000` retornava la mateixa longitud i també `X-Powered-By: Express`:

La interpretació va ser que Apache al port `80` actuava com a frontal o proxy, mentre que el port `3000` exposava directament l'aplicació Express. En tots dos casos s'arribava al mateix backend.

3.3 Extracció de rutes públiques

Es van extreure recursos visibles de la pàgina principal:

Rutes observades:

`robots.txt` no existia com a recurs vàlid:

`whatweb` va confirmar la lectura tecnològica:

3.4 Tancament de WEB-BASE inicial

L'aplicació no mostrava cap CMS, ni cap login, ni cap panell autenticat. Tampoc apareixien descàrregues ni artefactes públics. La senyal rellevant era una altra: la mateixa aplicació es presentava com una plataforma per executar codi Node.js dins d'un sandbox.

El text visible indicava:

Això va canviar el focus. La web deixava de ser una superfície genèrica i passava a ser una aplicació que executa codi no fiable.

4. Identificació de la tecnologia de sandbox

4.1 Descàrrega de pàgines informatives

Es van guardar les rutes públiques rellevants:

```
-o content/web/about.html  
ations -o content/web/limitations.html  
r -o content/web/editor.html
```

La raó de guardar aquestes pàgines era conservar evidència local i poder cercar termes tècnics sense dependre del navegador.

4.2 Cerca de termes clau

Es van cercar indicadors relacionats amb execució de codi, sandboxing i mòduls de Node.js:

La pàgina `/about` identificava la tecnologia de sandbox:

A més, enllaçava a:

Aquesta va ser una senyal de versió molt forta, encara que cal descriure-la amb precisió: és una versió observada des de l'aplicació, no una versió confirmada des del sistema de fitxers.

4.3 Restriccions declarades per l'aplicació

La pàgina `/limitations` indicava que alguns mòduls estaven restringits:

També llistava mòduls permesos:

Aquesta informació ajuda a entendre el model defensiu de l'aplicació. No es tracta d'una execució Node.js sense controls; hi ha un intent explícit de bloquejar mòduls perillosos. Tanmateix, aquesta capa no demostra que el sandbox sigui segur.

4.4 Flux real de l'editor

El fitxer `editor.html` va revelar com s'envia el codi al backend:

El navegador codifica el contingut amb `btoa()`:

```
code''),value;
```

i l'envia a:

La superfície tècnica dominant passa a ser:

5. Validació benigna de l'endpoint /run

5.1 Per què validar abans d'explotar

Abans d'utilitzar una PoC pública, es va validar el flux de manera benigna. Aquest pas evita confondre un problema de format, Base64, JSON o endpoint amb un error d'exploació.

Es va preparar un codi innocu:

i es va codificar en Base64:

Sortida:

5.2 Prova contra /run

Es va reproduir des de terminal el que feia el navegador:

Resposta:

La mateixa prova va funcionar contra el port directe **3000**:

Resposta:

La lectura és clara: l'endpoint `/run` accepta JSON, espera `code` en Base64, executa el JavaScript dins del backend i retorna la sortida al camp `output`.

5.3 Validació de mòdul restringit

Després es va comprovar si la restricció sobre `child_process` era real. Es va enviar l'equivalent a:

codificat com:

Petició:

Resposta:

Aquest resultat no descarta la via d'exploació. Al contrari, aclareix el model defensiu: l'aplicació bloqueja importacions directes de mòduls perillosos i delega la seguretat real de l'aïllament en `vm2`.

La pregunta tècnica ja no és:

La pregunta passa a ser:

6. Candidata pública: escape de sandbox a vm2

La revisió de vulnerabilitats públiques associades a `vm2` **3.9.16** condueix a CVE-2023-30547, una vulnerabilitat d'escape de sandbox a `vm2`. Segons les notes de treball, aquesta candidata afecta versions fins a **3.9.16**, està relacionada amb una sanitització incorrecta d'excepcions i va ser corregida posteriorment.

En aquesta fase se separen tres nivells:

Fet verificat

L'aplicació utilitza `vm2` i enllaça a 3.9.16.

Inferència raonable

La versió en ús pot ser vulnerable a CVE-2023-30547.

Pendent fins a validació

La vulnerabilitat s'ha de provar a l'endpoint real `/run` i demostrar execució fora del sandbox.

7. Validació de l'escape de sandbox

7.1 Prova mínima amb id

La validació inicial es va fer des de la pàgina:

Es va enganxar una PoC pública adaptada per executar `id`:



La sortida va ser:

La importància d'aquesta prova és que `child_process` estava bloquejat per importació directa. Obtenir la sortida d'`id` demostra que l'execució ja no queda limitada al flux normal del sandbox i que s'ha assolit el context del procés Node.js del backend.

7.2 Preparació de la reverse shell

Es va crear un script local `rev.sh` a la màquina atacant:

Contingut:

Es va iniciar un servidor HTTP per servir l'script:

I un listener per rebre la connexió:

Abans d'executar la PoC va caldre confirmar la IP real de la interfície VPN:

Sortida rellevant:

Aquest control va evitar barrejar IPs a l'script i al `curl` de la PoC.

7.3 PoC per obtenir shell

A `/editor` es va executar la PoC modificada per descarregar i executar `rev.sh`:



Al listener es va rebre connexió des de la màquina víctima:

Es va establitzar mínimament la shell:

Validació de context:

Sortida:

La fase d'accés inicial queda tancada: l'usuari inicial és `svc`.

8. Enumeració local després del foothold

8.1 Identificació d'usuaris interactius

Es va revisar `/etc/passwd` per detectar usuaris locals amb shell:

Usuaris rellevants:

Aquesta sortida va orientar el moviment lateral: si l'aplicació emmagatzemava credencials o hashes, `joshua` era l'usuari local candidat.

8.2 Cerca d'artefactes web

La bitàcola de referència apuntava a `/var/www/contacts`, però aquesta ruta no existia a la instància treballada. En lloc d'insistir sobre una ruta no verificada, es va enumerar `/var/www`:

Sortida rellevant:

La ruta correcta era:

Aquest ajust és metodològicament important: la referència orienta, però l'evidència de la màquina mana.

8.3 Confirmació de la base SQLite

Es va revisar el directori real:

Sortida:

La màquina tenia `sqlite3` instal·lat, així que no va caldre transferir la base per llegir-la.

9. Extracció de hash des de tickets.db

9.1 Enumeració de taules

Es van llistar les taules:

Sortida:

La taula `users` va passar a ser prioritària, perquè podia contenir credencials o hashes reutilitzables.

9.2 Volcat de la taula users

Es va mostrar esquema i contingut:

Sortida:

Contingut:

El prefix `$2a$12$` va permetre classificar el hash com a bcrypt.

9.3 Craqueig offline

El hash es va guardar a la màquina atacant:

Es va llançar Hashcat en mode bcrypt:

Resultat:

Credencial obtinguda:

La contrasenya es va craquejar offline, no a la víctima. Això redueix soroll al sistema objectiu i manté la fase de credencials separada de l'enumeració local.

10. Moviment lateral per SSH

Des de la fase 1 se sabia que SSH estava obert:

Es va provar la contrasenya recuperada contra l'usuari local `joshua`:

Contrasenya:

Accés confirmat:

Es va llegir la flag d'usuari:

Sortida:

La cadena de moviment lateral queda tancada:

11. Enumeració sudo com joshua

11.1 Revisió de permisos sudo

La primera comprovació d'escalada local va ser:

Contrasenya utilitzada:

Sortida rellevant:

La prioritat va canviar: ja no importava la base SQLite ni la web. La via principal era ara un script custom executable com `root`.

11.2 Inspecció de l'script abans d'executar-lo

Es van revisar permisos, tipus i contingut:

Permisos:

Tipus:

Contingut rellevant:

Després utilitza la contrasenya real en comandes MySQL:

i:

12. Anàlisi de la vulnerabilitat de l'script

L'script conté dues debilitats útils encadenades.

12.1 Comparació insegura en Bash

La línia crítica és:

```
if [[ $PASSWORD == $REAL_PASSWORD ]] then
```

En Bash, dins de `[[ ... == ... ]]`, el costat dret de la comparació es pot comportar com un patró. Si s'introdueix `*`, pot coincidir amb qualsevol contrasenya.

Es va validar executant:

Quan l'script va preguntar:

s'hi va introduir:

Sortida:

El bypass quedava confirmat.

12.2 Exposició de contrasenya per arguments de procés

Després del bypass, l'script executa `mysql` i `mysqldump` amb la contrasenya en línia de comandes:

Això fins i tot genera advertiments:

L'advertiment és una pista excel·lent: si la contrasenya viatja com a argument del procés, pot observar-se temporalment amb una eina com `pspy`.

13. Captura de la contrasenya real amb pspy

13.1 Preparació a la màquina atacant

Es va descarregar `pspy64s`:

13.2 Transferència a la víctima

Des de la sessió SSH com `joshua`:

`pspy` es va deixar corrent en una sessió SSH.

13.3 Segona sessió SSH per disparar l'script

Es va obrir una altra sessió SSH com `joshua` i es va executar:

És important distingir els dos peticions de contrasenya:

Aquí va la contrasenya de `joshua`:

Després apareix el petició de l'script:

Aquí va el comodí:

13.4 Contrasenya capturada

`pspy` va capturar el procés `mysqldump`:

Contrasenya recuperada:

La línia confirma tres fets:

1. el procés s'executa com `UID=0`;

2. l'script utilitza la contrasenya real llegida des de `/root/.creds`;
3. aquesta contrasenya apareix exposada als arguments del procés.

14. Escalada final a root

Es va provar la reutilització de la contrasenya amb `su root`:

Contrasenya:

Validació:

Sortida:

La flag de root va ser:

15. Resum tècnic final

15.1 Cadena d'exploació



15.2 Flags

15.3 Credencials i secrets del cas

16. Lliçons reutilitzables

16.1 Llegir l'aplicació abans de buscar exploits

Codify ensenya que una web senzilla pot revelar la seva pròpia via d'atac si se'n llegeixen bé les pàgines. `/about`, `/limitations` i `/editor` van aportar més valor que un fuzzing genèric primerenc.

Patró reutilitzable:

16.2 Distingir bloqueig directe de seguretat real

El bloqueig de `child_process` no feia segura l'aplicació. Només impedia una via directa. La protecció real depenia de `vm2`, i la versió observada era vulnerable.

Lliçó:

16.3 Validar el flux abans d'explotar

Abans de la PoC, es va confirmar que `/run` acceptava JSON, requeria Base64 i retornava sortida a `output`. Aquesta validació va evitar errors d'interpretació.

Lliçó:

16.4 L'evidència real corregeix la ruta del writeup

La ruta esperada era `/var/www/contacts`, però la màquina tenia `/var/www/contact`.

Lliçó:

16.5 Les bases locals d'aplicació són pivots freqüents

`tickets.db` contenia el hash de `joshua`. Aquest patró és molt habitual després d'un foothold web: l'aplicació acostuma a tenir bases locals, fitxers `.env`, configuracions o artefactes que permeten moure's a un altre usuari.

16.6 El craqueig es fa offline

El hash `bcrypt` es va extreure i es va craquejar a la màquina atacant. No es va executar craqueig a la víctima.

Lliçó:

16.7 No executar scripts sudo sense llegir-los

`sudo -l` va mostrar un script custom. La via correcta va ser llegir-lo abans d'executar-lo. La fallada era a la seva lògica: lectura de `/root/.creds`, comparació insegura i ús de la contrasenya com a argument.

16.8 Els advertiments importen

El missatge:

no era soroll. Era una pista directa que la contrasenya podia veure's en processos.

16.9 Diferenciar peticions de contrasenya evita errors

A l'escalada hi havia dos peticions de contrasenya diferents:

```
-> spongebob1  
-> *
```

Confondre'ls produeix errors d'autenticació que no tenen a veure amb la vulnerabilitat.

16.10 Incidències operatives que no són troballes de la màquina

Durant el laboratori van aparèixer incidències locals útils per recordar, però no s'han de confondre amb vulnerabilitats de Codify:

- utilitzar `path` com a variable a `zsh` va trencar temporalment la resolució de comandes com `curl`, `grep` i `sort`;
- un `heredoc` amb blocs Markdown interns pot quedar mal copiat;
- el pegat brut a la shell es va corregir amb:

Aquestes incidències són útils per a l'aprenentatge operatiu, però no formen part de l'explotació de la màquina.